Thamill Indian B U
Cohort-15 (AI Engineer)

Day 10 Post-Session Assignment & Engineering Reflection

AI Engineering Program | Day 10: Enterprise Knowledge & Action Assistant
Repository: https://github.com/ThamillIndianBlend360/AI-Assignments

---

_

Part 1 - Output & Execution Evidence

The Day 10 assignment focused on implementing controlled function calling for an Enterprise HR Assistant using Ollama (llama3) and Pydantic validation. The system separates read-only lookups from state-mutating actions, enforcing an application execution boundary and human confirmation before any write operation.

Code Structure & Repository Link

All implementation files, test scripts, and execution logs have been pushed to the GitHub repository: **https://github.com/ThamillIndianBlend360/AI-Assignments/tree/main/Day10**

• Day10/main.py: Core HR Assistant implementing Pydantic tool schemas, read/write tool separation, human authorization prompt, and execution logging.
• Day10/day10_execution.log: Audit trail recording user queries, validation steps, authorization decisions, and tool outputs.

Execution Trace (Normal Path)

User Query: *"My user ID is E101. Please submit a 3-day leave request starting 2026-11-10 for a personal trip."*

```
2026-10-28 12:33:29,276 [INFO] User Query: My user ID is E101. Please submit a 3-day leave
request starting 2026-11-10...
💬 User Query: My user ID is E101. Please submit a 3-day leave request starting 2026-11-10 for
a personal trip.
🤖 Assistant Initial Response:
  Check leave balance: {"tool": "get_employee_leave_balance", "args": {"user_id": "E101"}}
⚙️ Model requested tool: get_employee_leave_balance
2026-10-28 12:34:24,637 [INFO] [READ-ONLY TOOL EXECUTION] get_employee_leave_balance for
user_id='E101'
🔍 Observation: {"user_id": "E101", "name": "Priya", "leave_balance": 12, "department":
"Engineering"}
🤖 Assistant Submitting Leave Request:
  Submit request: {"tool": "create_leave_request", "args": {"user_id": "E101", "start_date":
"2026-11-10", "days": 3, "reason": "Personal trip"}}
⚠️ SECURITY ACTION CONFIRMATION REQUIRED
Tool: create_leave_request
Proposed Mutation Arguments: {
  "user_id": "E101", "start_date": "2026-11-10", "days": 3, "reason": "Personal trip"
```

```
}
Do you authorize the application to execute this WRITE action? [y/N]: y
2026-10-28 12:34:26,102 [INFO] User APPROVED write action 'create_leave_request'
2026-10-28 12:34:26,105 [INFO] [WRITE TOOL EXECUTION] create_leave_request for E101: 3 days
from 2026-11-10
🔍 Observation: {"status": "SUCCESS", "message": "Successfully scheduled 3 day(s) leave for
Priya.", "remaining_balance": 9}
🏁 Final Response: Leave request submitted successfully! Your updated leave balance is 9 days.
```

Part 2 - Results & Observations

What Worked Successfully

1. Schema-Enforced Input Validation: Pydantic schemas successfully intercepted invalid types and out-of-bounds parameters (e.g., negative leave days or non-string user IDs) before tool invocation.
2. Application Execution Isolation: The LLM never directly executed Python code; it only generated structured JSON proposals that the Python application validated and executed.
3. Human-in-the-Loop Gatekeeping: Write operations (state mutations) were strictly paused until explicit interactive authorization [y/N] was granted by the user.

What Behaved Unexpectedly

Local Model Output Hallucination in Single-Turn Simulation: During testing with local llama3, the model occasionally attempted to simulate both the tool request and the fictitious observation within a single assistant response turn (e.g., outputting 'According to the response, your current leave balance is 5 days' before the tool had actually executed). This required strict parsing logic in parse_step() to extract tool proposals and suppress model-simulated output.

Part 3 - Engineering Reasoning

Key Architectural Decisions & Trade-Offs

• **Decision:** Decouple Tool Calling from Model Authority.
• **Alternative Considered:** Allowing the LLM to directly invoke API endpoints via autonomous agent loops.
• **Trade-off Accepted:** Slightly higher application latency due to two LLM turns and confirmation prompts, in exchange for absolute execution safety and 0% unauthenticated state mutation risk.

• **Decision:** Strict Pydantic Input Guardrails.
• **Alternative Considered:** Passing raw JSON dictionaries straight to tool functions.
• **Trade-off Accepted:** Validation failures return structured error observations to the LLM, consuming context window tokens but preventing database corruption or unexpected application crashes.

Thamill Indian B U
Cohort-15 (AI Engineer)

Part 4 - FDE & Customer Reflection

1. Why the model should NOT own direct execution authority

Large Language Models are non-deterministic pattern generators. Giving an LLM direct execution authority over database mutations or external APIs creates severe vulnerabilities—such as prompt injection attacks, accidental duplicate requests, or parameter hallucination. The LLM must act purely as an intent parser that proposes actions, while the deterministic application software validates permissions, parameters, and business logic before executing.

2. Pre-Deployment Discovery & Customer Validation Questions

Before deploying tool-calling assistants with an enterprise customer, an Forward Deployed Engineer (FDE) must validate:

• RBAC & Identity Alignment: How does the assistant verify that user 'E101' has authorization to perform actions on behalf of that employee ID?

• Idempotency & Reversibility: Are API endpoints idempotent, or can duplicate tool calls create redundant leave requests or double-billing?

• Fail-Safe Rollbacks: What is the transaction rollback policy if a downstream API fails midway through execution?

3. Business Evidence for Architecture Upgrades

Upgrading from basic function calling to complex multi-agent orchestrations should only be triggered when business metrics demand it—such as when tool call error rates exceed 5% due to parameter complexity, or when tool execution chains require multi-system transactional rollbacks beyond single-prompt capabilities.